

Disentangled Representation Learning for Structural Role Discovery

Wang Zhang , Lin Pan , Xuan Guo , and Pengfei Jiao , *Member, IEEE*

Abstract—Roles are defined as the equivalent classes of isomorphic nodes in the network. They focus on the local connective patterns and describe the structural similarities between nodes, and learning role-based network embeddings can help to recognize identities or functions of entities in real-world networks. This field has been studied over the past decades, however, the existing role-based network embedding methods all concentrate too much on distinguishing node structures and ignore that nodes can belong to different roles even if they have the same local structures. Not only the classical network structure can influence the role of node, but also the emergence of different type of relationship between nodes has impact to it. We believe that roles are influenced by multiple independent factors, and nodes in the same roles should possess both similar local structures and entangled patterns with neighbors hidden in each factor, but most of the existing methods did not clearly point out this challenge. Therefore, we propose a disentangled framework based on graph neural networks to simultaneously model the local structures and interactive relationships in multiple factors to generate role-oriented network embeddings. We design a novel role-aware neighbor choosing mechanism to assign each neighbor a different interactive weight for each latent factor, which can measure its influence on roles. We conduct experiments on synthetic and real-world networks, and the results demonstrate the superiority and effectiveness of our model.

Index Terms—Disentangled model, network embedding, role-aware neighbor choosing, structural role.

I. INTRODUCTION

ROLES in the network describe the structural similarities between nodes [1]. It was first defined as the equivalent class of isomorphic nodes [2], such as the center node, leaf

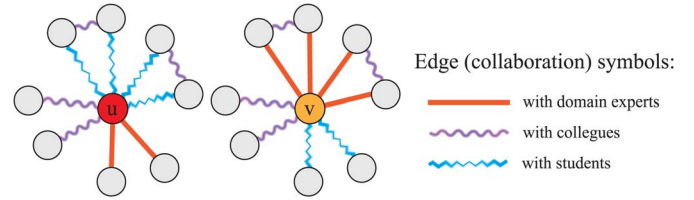


Fig. 1. Example of collaborative network of academic authors.

node, and bridge node. As a fundamentally different but complementary concept compared with community, role is independent of network distance or density, and nodes that have similar local connective patterns would share the same role [3]. It describes the function of node in the network structure, and can reflect real-world identity status to some degree. With the development of network embedding (also known as network representation learning), learning role-oriented network representations has plenty of applications for downstream tasks, such as node or graph classification [4], link prediction [5], etc. For more details of role discovery, it can be found in the survey [3], [6].

Most of the existing network embedding methods are designed for the community [7]. In other words, these methods focus on preserving the node proximity in the embedding space, such as DeepWalk [8] and graph convolutional network (GCN) [9]. They map nodes that are close or densely connected into similar positions in the embedding space, but fail to the task of role discovery [10]. There are a few methods of generating role-based network representations to measure structural similarities. For example, struc2vec [11] leverages the Fast-dynamic time warping (DTW) [12] method to compute node-pair similarities and constructs a multilayer complete graph. Then they use the random walk on it to generate role-oriented node representation. There are also other methods that leverage a variety of structural features or similarities for role discovery [13], [14].

However, these role-oriented network embedding methods consider too much into distinguishing local connective patterns of nodes. They ignore that in real-world networks, even if nodes have similar local structures, their roles can be different. Consider such a situation, as shown in Fig. 1, it exhibits two nodes with the same local structures in the collaborative network of academic authors, where roles denote their academic influences. The two authors both have three interactive relationships with their neighbors, but node v prefers to cooperate with domain

Manuscript received 6 July 2023; revised 5 March 2024 and 16 April 2024; accepted 21 May 2024. Date of publication 17 June 2024; date of current version 3 December 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62372146, in part by Zhejiang Provincial Natural Science Foundation of China under Grant LDT23F01015F01, in part by the Fundamental Research Funds for the Provincial Universities of Zhejiang under Grant GK229909299001-008, in part by Hainan Province Science and Technology Special Fund under Grant ATIC2023030001, and in part by Hangzhou Artificial Intelligence Major Scientific and Technological Innovation Project under Grant 2022AIZD00161. (Wang Zhang and Lin Pan contributed equally to this work.) (Corresponding author: Pengfei Jiao.)

Wang Zhang and Xuan Guo are with the College of Intelligence and Computing, Tianjin University, Tianjin 300350, China (e-mail: wangzhang@tju.edu.cn; guoxuan@tju.edu.cn).

Lin Pan is with the School of Marine Science and Technology, Tianjin University, Tianjin 300072, China (e-mail: linpan@tju.edu.cn).

Pengfei Jiao is with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou 310018, China (e-mail: pjiao@hdu.edu.cn).

Digital Object Identifier 10.1109/TCSS.2024.3404975

experts, while node u prefers to cooperate with his/her students. Although nodes u and v have similar local structures, their roles can be different because of the interactive relationships. We believe that the roles of nodes are influenced by entangled patterns of neighbors with multiple independent factors. And to solve the challenge, methods should possess the power of disentangling these latent factors. However, all the existing role-based methods cannot make it.

More recently, disentangled representation learning has attracted great attention, particularly in the field of image representation learning [15]. It aims to learn representations that separate the explanatory factors hidden in the observed data [16]. The disentangled mechanism can be more robust and less sensitive to the misleading correlations presented in the limited training data [17], and enhance the interpretability of representations [18]. These methods are not designed for the irregular structure of graph data, and how to disentangle the latent factors with the graph neural network (GNN) remains largely unexplored.

The complex formation process of graphs requires the GNNs to have the ability to infer the latent factor that may cause an edge, based on limited information, such as node attributes or structures. For instance, DisenGCN [19] designs a neighbor routing mechanism, aggregating the neighbors that are most similar to the central node in each latent factor. Macrid variational auto-encoder (VAE) [20] achieves macro disentanglement by inferring the high-level concepts associated with user intentions, while capturing the preference of a user regarding the different concepts separately.

However, none of the above disentangled methods can be applied to the task of role discovery. According to the definition, nodes with the same roles can be distant from each other, and the roles of neighbors and center nodes are usually different. Thus, it is unfaithful to simply aggregate similar neighbors or assign large interactive weights for them in some factors, but the existing disentangled GNN framework still focuses on aggregating neighbors that are similar to the center node, and cannot simultaneously capture the local structure of node and model the complex entangled patterns with neighbors. To address the above shortcoming, we propose a disentangled GNN framework (DRSR) to learn role-based node representations. To the best of our knowledge, we are the first to design the disentangled representation learning framework to enhance role discovery. Specifically, we design a novel role-aware neighbor choosing mechanism, which adaptively assigns different weights for neighbors in each latent factor to disentangle the interactive patterns related to roles of nodes. Our contribution can be summarized as follows.

- 1) We extend the definition of role equivalence that other existing methods have not done. We argue that nodes with the same roles should have not only equivalent local structures but also entangled patterns of neighbors with respect to multiple latent factors.
- 2) We design a disentangled framework for role-oriented node representations. We propose a disentangled GNN framework DRSR including the novel role-aware neighbor choosing mechanism, which can simultaneously

capture the local structure of node and model the complex entangled patterns with neighbors.

- 3) We conduct sufficient experiments on both synthetic and real-world networks. We compare our model with state-of-the-art methods, and the results demonstrate the correctness of our hypothesizes, along with the effectiveness and superiority of our model.

II. RELATED WORK

A. Role-Orient Network Embedding

Role-oriented network embedding methods aim to map structurally similar nodes into close positions in the embedding space. The random walk is the classical method of generating network embeddings. For example, node2vec [21] improves the traditional random walk leveraged by DeepWalk [8], and introduces two hyperparameters to control the directions of walks. Then it leverages the language model Skip-Gram [22] to generate node representations. Struc2vec applies the Fast-DTW to compute the structural similarities of nodes, and constructs a multilayer complete network where the weights of edges are the similarities. There are also some feature-based random walk methods, such as Role2vec [5], RiWalk [23], and NODE2BITS [24].

Some network embedding methods decompose the structural similarities or features for role discovery. Role extraction (RoIX) [25] leverages the nonnegative matrix factorization (NMF) on the features extracted by recursive feature extraction (ReFeX) [26], which recursively aggregates the local and ego-net features to perceive the network structure. Guided learning for role discovery (GLRD) [27] and RID ϵ Rs [28] also use the features to generate node representations. GraphWave [29] applies the graph wavelet diffusion to measure structural similarity and structural embeddings using graph kernels (SEGK) [30] leverages the graph kernels to distinguish local structure. Struc2gauss [31] applies the RoleSim method to construct a similarity matrix between roles and optimizes a kullback-leibler (KL)-energy function to learn Gaussian embeddings.

Deep recursive network embedding (DRNE) [32] is the first to introduce the deep learning model into role-based network embedding. It uses LSTM to aggregate neighbors and add the node degree regularizer to preserve local structure. Graph auto-encoder guided by structural information (GAS) [33], deep mutual encode for embedding (DMER) [34], and role-based network embedding via structural features reconstruction with degree-regularized constraint (RESN) [35] leverage the ReFeX features and construct the encoder-decoder framework to reconstruct structural features. Curvature-based network embedding with stochastic equivalence (CNESE) [36] uses the network curvature [37] to describe network structure. Role discovery based on autoencoder and attention (RDAA) [14] captures the neighbor dependencies through the self-attention mechanism. Adversarial learning between higher-order and local features (ARHOL) [13] tries to integrate higher-order and local features into an adversarial framework to learn role-oriented node embeddings. But these methods concentrate too

much on network structure, without considering the complex entangled patterns of neighbors which can influence node roles.

B. Disentangled Representation Learning

Disentangled representation learning aims to identify and disentangle the underlying explanatory factors [18]. β -VAE [38] demonstrates that disentanglement can emerge once the KL divergence regularizer is properly adjusted. MacridVAE [20] achieves macro disentanglement by inferring the high-level concepts associated with user intentions, while capturing the preference of a user regarding the different concepts separately. It learns disentangled representations of user behavior, and is designed for social recommendation. Though there are a few approaches which propose the disentangled GNN framework, such as DisenGCN [19], which designs a neighbor routing mechanism and aggregates the neighbors that are most similar to the central node for each independent latent factor, they all focus on choosing the similar neighbors to aggregate. But for the task of role discovery, nodes with the same roles are usually indirectly connected, and the roles of the center node and its neighbors are also different in general. We assume that nodes of equivalent roles should possess equivalent entangle patterns with neighbors, and concentrate on adaptively learning the interactive weights with respect to multiple latent factors.

III. METHOD

A. Notions

We introduce the basic notations in our article in this section. A network can be represented as: $G = (V, E)$, where V is the set of nodes, and $E \subseteq V \times V$ is the set of edges among nodes. The neighbors of the node u are defined as $N(u) = \{v | (u, v) \in E\}$. The adjacency matrix is denoted as $\mathbf{A} \in \mathbb{R}^{|V| \times |V|}$. The one-hop ego-net of node u is defined as $\mathcal{G}_u = \{V(g_u), E(g_u)\}$, where $V(g_u) = \{u\} \cup \{v \in V | (u, v) \in E\}$, and $E(g_u) = \{(v_i, v_j) | (v_i, v_j) \in E, v_i \in V(g_u), v_j \in V(g_u)\}$. The detailed description of symbols can be seen in Table I. Before introducing the details of our proposed model, we first declare some definitions and hypothesizes. As shown in Fig. 1, the role of the center node is influenced by its neighbors along with several hidden factors, so we declare the first definition of the interactive weight with neighbor.

Definition 1 (Interactive Weight): For the specific node $u \in V$, we define the interactive weight with its neighbor v in the factor k as the set $p_{uv,k}$, where $k = 1, 2, \dots, K$ and K denotes the number of factors.

Given the factor k , the interactive weight $p_{uv,k}$ measures the importance of the neighbor v to the role of the center node u . To extend the concept of role equivalence as described above, we declare the following definition of entangled pattern to take the interactive weights in all factors into consideration.

Definition 2 (Entangled Pattern): Given node $u \in V$ and factor k , the set of its interactive weights can be denoted as $\mathbb{P}_k(u) = \{p_{uv,k} | (u, v) \in E\}$, then we define the entangled pattern of node u as the sequence $Ep(u) = (\mathbb{P}_1(u), \mathbb{P}_2(u), \dots, \mathbb{P}_K(u))$.

TABLE I
SYMBOLS AND DEFINITIONS

Symbol	Definition
G	The specific graph
V	The set of nodes in G
E	The set of edges in G
$N(u)$	Neighbors of node u in G
$\mathbf{A}$	The adjacency matrix of G
$\mathcal{G}_u$	The one-hop ego-net of node u
$V(g_u)$	The set of nodes in $\mathcal{G}_u$
$E(g_u)$	The set of edges in $\mathcal{G}_u$
K	The number of latent factors
$p_{uv,k}$	The interactive weight between node u and node v with respect to the k th latent factor
$\mathbb{P}_k(u)$	The set of interactive weights of node u to all its neighbors with respect to the k th latent factor
$Ep(u)$	The defined entangled pattern of node u
$R(u)$	The role of node u
$\mathbf{I}_n$	The identity matrix with n rows
$\mathbf{x}_u$	The input features of node u
$\mathbf{z}_{u,k}$	The representation of node u with respect to the k th latent factor
$\mathbf{z}_{u,k}^{(l)}$	The representation of node u with respect to the k th latent factor at the l th layer
$\mathbf{z}_u$	The final output representation of node u
d_{in}	The dimension of the input features
d_{out}	The dimension of the output representations
$d_k^{(l)}$	The dimension of the hidden representations with respect to the k th factor and l th layer
L	The number of model layers
$\mathbf{h}_{u,k}^{(l)}$	The hidden representation of node u before aggregation
$\mathbf{W}_k^{(l)}, \mathbf{b}_k^{(l)}$	The weight matrix and bias of the l th disentangled encoder layer
$\mathbf{Q}_k^{(l)}$	The trainable parameter that measures influences neighbors to roles with respect to the l th encoder layer
$\mathbf{c}_{u,k}^{(l)}$	The role coefficient vector of node u with respect to the k th latent factor and l th layer
$p_{uv,k}^{(l)}$	The calculated interactive weight between node u and node v with respect to the k th latent factor and l th layer
$f_1, \dots, f_6$	The extracted features of the one-hop ego-net
$\hat{\mathbf{h}}_u^{(l')}$	The hidden representation of node u with respect to the l' th decoder
$\hat{\mathbf{W}}^{(l')}, \hat{\mathbf{b}}^{(l')}$	The weight matrix and bias of the l' th decoder
$\mathbf{s}_u$	The extracted structural features of node u
$\hat{\mathbf{s}}_u$	The reconstructed structural features of node u by decoder
$\mathcal{L}$	The loss function of model

As shown in Fig. 1, there are three factors (cooperation with domain experts, colleagues, and students) contributing to the roles of the center node u and v , and the reason of their different roles is they have different entangled patterns. In other words, the set of interactive weights are different for any factor (node u prefer to cooperate with students but node v not). Thus, to extend the role equivalence, we assume that nodes in the same role should have both same interactive weights and same entangled patterns, and propose the following hypothesizes.

Hypothesis 1: Factor k is likely to be the reason to determine the interactive weights of the neighbors, and nodes that are role equivalent should have equivalent interactive weights w.r.t. k . We use $R(u)$ to denote the role of node u , then we have $R(u) = R(v) \iff \mathbb{P}_k(u) = \mathbb{P}_k(v)$.

The Hypothesis 1 is derived from the traditional structural equivalence [2]. When considering the specific factor k

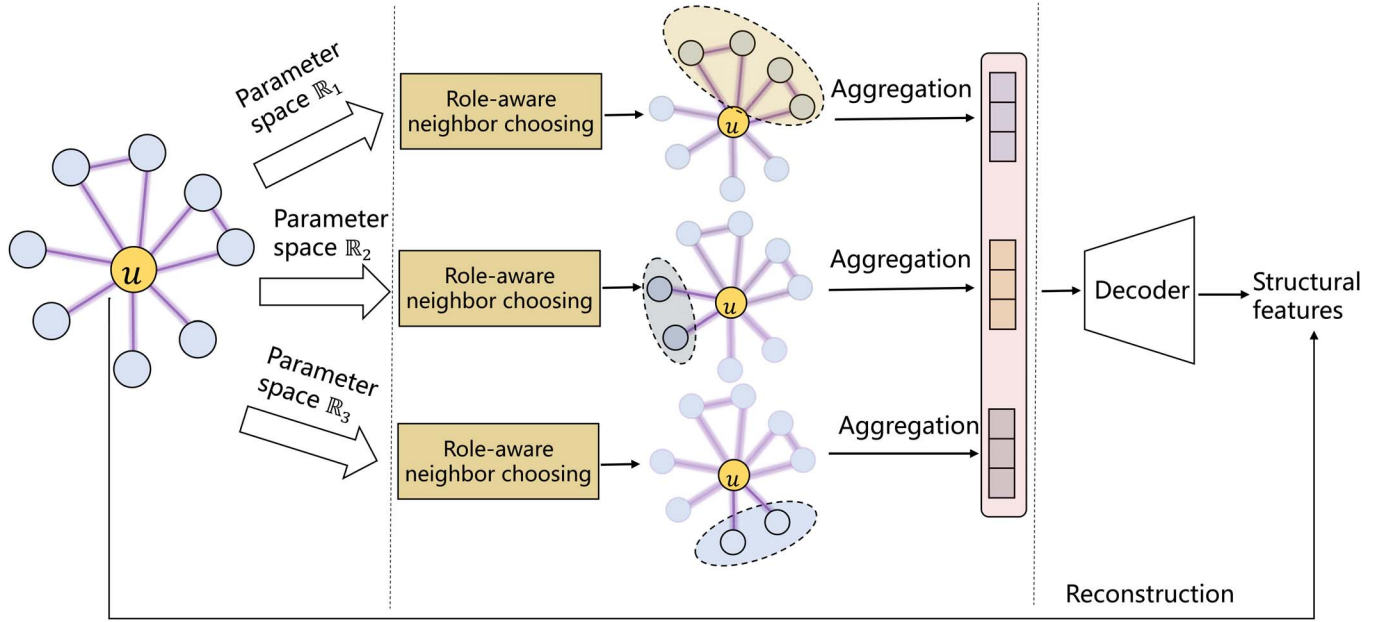


Fig. 2. Illustration of the proposed model DRSR.

(similar to traditional equivalence without the definition of factor), nodes in the same role should have same local connective patterns [2]. Then we extend it to all factors and propose the following hypothesis.

Hypothesis 2: Nodes that are role equivalent should have same entangled patterns, and we have $R(u) = R(v) \iff Ep(u) = Ep(v)$. Note that the entangled pattern $Ep(u)$ is a sequence which is sensitive to order, we can infer that $R(u) = R(v) \iff \mathbb{P}_k(u) = \mathbb{P}_k(v)$ for all $k = 1, 2, \dots, K$.

According to the classical definition of roles and our two hypothesizes, we argue that roles are determined by multiple latent factors, and nodes in the same role should possess both equivalent local structures and entangled patterns. In addition, if two nodes that are structurally similar but have different interactive relationships with neighbors, their roles should be different.

B. Model

In this section, we will introduce the proposed model DRSR in detail. As shown in Fig. 2, the model consists of three parts. For the given node u , we first map the node features $\mathbf{x}_u$ into different parameter spaces with respect to K factors, and then for each k , we design the role-aware neighbor choosing mechanism to adaptively assign interactive weights for neighbors. The weights of neighbors are different for each factor and this process can disentangle the entangled patterns to describe the influences of neighbors on center node roles. Finally, we combine the node representations of K factors and reconstruct some structural features via a shared decoder.

1) *The Disentangled Encoder Layer:* Given the node features $\mathbf{x}_u \in \mathbb{R}^{d_{in}}$ and $\{\mathbf{x}_v | (u, v) \in E\}$ as the inputs of our model, the encoder outputs the node representation of node u as $\mathbf{z}_u$, and we have $\mathbf{z}_u = [\mathbf{z}_{u,1}, \mathbf{z}_{u,2}, \dots, \mathbf{z}_{u,K}]$, where $\mathbf{z}_u \in \mathbb{R}^{d_{out}}$ and

$\mathbf{z}_{u,k} \in \mathbb{R}^{d_{out}/K}$, while d_{in} and d_{out} denote the dimension of input and output, respectively. The model consists of K channels, and we make sure to randomize the parameters to map the input features into different parameter spaces

$$\mathbf{h}_{u,k}^{(l)} = \sigma(\mathbf{W}_k^{(l)T} \mathbf{z}_{u,k}^{(l-1)} + \mathbf{b}_k^{(l)}), l = 1, 2, \dots, L \quad (1)$$

where $l = 1, 2, \dots, L$ is the current layer and L is the number of layers. $\mathbf{W}_k^{(l)}$ and $\mathbf{b}_k^{(l)}$ denote the weight and bias of the l th layer hidden in the k th factor, respectively. For the factor k , $\mathbf{h}_{u,k}^{(l)}$ is the hidden representation of node u before aggregation and $\mathbf{z}_{u,k}^{(l)}$ denotes the output of the l th layer with $\mathbf{z}_{u,k}^{(0)} = \mathbf{x}_u$. We take the identity matrix $\mathbf{I}_n$ as input for all the factors with $n = |V|$, so $\mathbf{x}_u$ is the one-hot encoding of node u . $\sigma(\cdot)$ is the nonlinear activation function to capture the complex relationships between layers.

The obtained vector $\mathbf{h}_{u,k}^{(l)}$ approximately describes the aspect of node u that are related to the k th factor. It may contains meaningful information about the related aspects. But the role of node is dependent on the neighbors, and this vector cannot surely represents the role even with deep layer. It is worth noting that our disentangled method is similar to the multihead methodology, but they have significant difference, which will be introduced in the next part.

2) *Role-Aware Neighbor Choosing:* In order to disentangle the entangled patterns of neighbors hidden in the multiple factors, we aim to learn different interactive weights with neighbors for each factor k . Specifically, we design the role-aware neighbor choosing mechanism based on the two hypothesizes. Some existing approaches have described the relationships [19], but they focus on aggregating neighbors that are similar to center nodes in each independent factor. While for role-oriented network embedding methods, it is crucial that nodes in the same role are usually indirectly connected, and the roles of neighbors

and center nodes are different. It is more complex to measure the influences of neighbors to the role of the center node.

Thus, we introduce the parameter $\mathbf{Q}_k^{(l)}$ with respect to the encoder layer l and the factor k to measure the influences of neighbors to roles. Given node u and the hidden representation $\mathbf{h}_{u,k}^{(l)}$, we compute the role coefficient vector for node u that the current encoder can perceive as follows:

$$\mathbf{c}_{u,k}^{(l)} = \mathbf{Q}_k^{(l)T} \mathbf{h}_{u,k}^{(l)} \quad (2)$$

where $\mathbf{c}_{u,k}^{(l)}$ is the role coefficient vector. Note that we apply different $\mathbf{Q}_k^{(l)}$ for different layers and factors, because it measures the local structure that the current encoder layer can perceive. And with increase of the number of encoders, the model can capture different local structure. What we need to do next is assigning weights that measure the influences of neighbors and aggregating the neighborhood to represent role.

We use the inner product between neighbors' hidden representations and role coefficient to measure the weights of the neighbors

$$\begin{aligned} p_{uv,k}^{(l)} &= \text{softmax}(\mathbf{h}_{v,k}^{(l)} \mathbf{c}_{u,k}^{(l)T} / d_k^l) \\ &= \frac{\exp(\mathbf{h}_{v,k}^{(l)} \mathbf{c}_{u,k}^{(l)T} / d_k^l)}{\sum_{k'=1}^K \exp(\mathbf{h}_{v,k'}^{(l)} \mathbf{c}_{u,k'}^{(l)T} / d_k^l)} \end{aligned} \quad (3)$$

where $p_{uv,k}^{(l)}$ is the influence of the neighbor v to the role of center node u hidden in the k th factor with respect to the l th encoder layer. Note that this part is the significantly different to the traditional multihead method based on sequential data, where we do the softmax among all channels, while the multihead attention concentrates on each separate channel. In other word, we have done an attention-like operator on the orthogonal direction of multihead attention, which can make the node representations become disentangled and address the challenges and example in our introduction, and the effectiveness is verified in our experiments (Sections IV-G and IV-I).

Then we use d_k^l to denote the dimension of the l th layer and avoid the results to become too large. Since the different encoder layers can perceive different local structures, the influences of neighbors become different in the same way. Then we are able to properly choose neighbors and aggregate them to represent role of node u as follows:

$$\mathbf{z}_{u,k}^{(l)} = \mathbf{h}_{u,k}^{(l)} + \sum_{v \in N(u)} p_{uv,k}^{(l)} \mathbf{h}_{v,k}^{(l)}. \quad (4)$$

Finally, we can obtain the representation of node u as the combination of the outputs of the last encoder layer

$$\mathbf{z}_u = [\mathbf{z}_{u,1}^{(L)}, \mathbf{z}_{u,2}^{(L)}, \dots, \mathbf{z}_{u,K}^{(L)}]. \quad (5)$$

3) *Decoder*: The encoder of our model DRSR aims to disentangle the latent factors and describes the different interactive relationships of neighbors. Then the decoder needs to leverage some structural information to make sure that the disentanglement of the encoder is related to role and local structure. It is known that two nodes u and v tend to share the same role if they have similar local structure, however, though the

adjacency matrix of graph contains the information of network topology, it is hard to compare the structural roles of two nodes, as the adjacency matrix is generally sparse and the nodes in the same role may be far away from each other. It is essential to extract effective local features from adjacency relationships instead of simply reconstructing adjacency matrix. Here, we will introduce our methodology of extracting structural features. Specifically, we extract one local feature and five ego-net-based features for each node. For the node u , the basic features based on $\mathcal{G}_u$ are defined as follows.

- 1) Number of edges in $\mathcal{G}_u$: $f_1 = |E(\mathcal{G}_u)|$.
- 2) Sum of degree for nodes in $V(\mathcal{G}_u)$: $f_2 = \sum_{u \in V(\mathcal{G}_u)} d(u)$.
- 3) The proportion of edges in $\mathcal{G}_u$ to all edges within and leaving $\mathcal{G}_u$: $f_3 = f_1 / f_2$.
- 4) The proportion of edges leaving $\mathcal{G}_u$ to all edges within and leaving $\mathcal{G}_u$: $f_4 = (f_2 - f_1) / f_2$.
- 5) The degree of node u : $f_5 = |N(u)|$.
- 6) The clustering coefficient for node u .

The above features can denote the local structure, and to make them perceive deeper structural information, we recursively aggregate the features of center node and its neighbors and compute the mean and sum values in each iteration with cutting off some unrelated ones similar to ReFeX [26]. In addition, to avoid large differences, the feature values are mapped to 0 or 1 via vertical logarithmic binning. Finally, we use the multilayer perceptrons (MLP) to reconstruct them. We leverage these features because of the efficiency, which can also be replaced by any structural information related to local structures.

We use $\mathbf{S}$ to denote the structural features, and the features of node u is denoted as $\mathbf{s}_u$. Then the decoder is defined as follows:

$$\hat{\mathbf{h}}_u^{(l')} = \sigma(\hat{\mathbf{W}}^{(l')T} \hat{\mathbf{h}}_u^{(l'-1)} + \hat{\mathbf{b}}^{(l')}), l' = 1, 2, \dots, L' \quad (6)$$

where L' is the number of the decoder. $\hat{\mathbf{W}}^{(l')}$ and $\hat{\mathbf{b}}^{(l')}$ denote the weight and bias of the l' th decoder, respectively. $\hat{\mathbf{h}}_u^{(0)} = \mathbf{z}_u$ is the input of the decoder and $\hat{\mathbf{s}}_u = \hat{\mathbf{h}}_u^{(L')}$ is the reconstructed features of node u . Finally, the loss function of the whole model is defined as follows:

$$\mathcal{L} = \sum_{u \in V} \|\mathbf{s}_u - \hat{\mathbf{s}}_u\|^2. \quad (7)$$

The pseudocode of our model is shown in Algorithm 1.

C. Complexity Analysis

Let $|V|$ denote the number of nodes, $|E|$ denote the number of edges, and f denote the dimension of the structural features $\mathbf{S}$. Given the number of latent factors K , we define the dimensions of the inputs and representations as d_{in} and d_{out} , then the dimension of the output for each factor k is d_{out}/K . For each node u and factor k , the complexity of mapping node features into different parameter spaces is $O(d_{\text{in}} d_{\text{out}}/K)$, and the process of computing role coefficient vectors costs $O(d_{\text{out}}^2/K^2)$. Assigning weights for all neighbors costs $O(|N(u)| d_{\text{out}})$, in other words, each aggregation costs $O(|N(u)| d_{\text{out}})$. Thus, for all nodes and factors, each layer of encoder takes $O(|V|(d_{\text{in}} d_{\text{out}} + d_{\text{out}}^2/K) + |E|K d_{\text{out}})$. Note that the input of our model is the one-hot encoding, which is a sparse matrix, and K is a small

TABLE II
STATISTICAL INFORMATION OF THE REAL-WORLD NETWORKS

Dataset	# Nodes	# Edges	# Average Clustering	# Density (in Percent)	# Labels
Brazil	131	1003	0.636	11.779	4
Europe	399	5993	0.539	7.548	4
United States	1190	13 599	0.501	1.922	4
Actor	7758	26 646	0.079	0.089	4
Aminer-network	67 667	186 863	0.623	0.008	3
Aminer-data	98 460	303 029	0.634	0.006	3
Aminer-system	146 092	400 863	0.612	0.004	3

Algorithm 1 The optimization algorithm for the DRSR

Input: $G = (V, E)$: the network;
 K : the number of hidden factors;
 S : the structural features in the decoder.
Output: Node embeddings $\{z_i\}_{i=1}^n$.

- 1: Randomly initialize K parameters to map node features into different spaces;
- 2: **for** each epoch **do**
- 3: **for** each node u **do**
- 4: Map the features x_u into the hidden space via Eq. 1;
- 5: Compute the interactive weights with neighbors $p_{uv,k}^{(l)}$ for all hidden factors via Eq. 2 and Eq. 3;
- 6: Aggregate neighbors to generate node representations z_u via Eq. 4 and Eq. 5;
- 7: Leverage the z_u to reconstruct structural features S via Eq. 6;
- 8: **end for**
- 9: Compute the loss function $\mathcal{L}$ via Eq. 7;
- 10: Update all the parameters with gradient backpropagation;
- 11: **end for**

value, then the complexity is $O(|V|d_{\text{out}}^2 + |E|d_{\text{out}})$. As for the encoder, we reconstruct the structural features $S \in \mathbb{R}^{|V| \times f}$, so the decoder takes $O(|V|(d_{\text{out}}f))$. The complexity of the whole model is $O(|V|d_{\text{out}}^2 + (|V|f + |E|)d_{\text{out}})$. As d_{out} and f are usually much less than the number of nodes and edges, the proposed model DRSR has equivalent complexity to the commonly used GNNs.

IV. EXPERIMENT

A. Dataset

We conduct extensive experiments on several real-world networks, where the labels of nodes are related to their roles or function. The statistical information can be seen in Table II, and the detailed introduction is as follows.

- 1) *Air-traffic network* [11]: There are three networks, consisting of American, Brazilian, and European air-traffic networks (shortly denoted as United States, Brazil, and Europe). In these networks, nodes represent airports and edges represent the existing flights between airports. The airports are labeled based on the number of airplanes or people.

- 2) *Actor co-occurrence network* [23]: It is an actor-occurrence network (shortly denoted as Actor) extracted from a large one [39]. In the Actor network, nodes represent actors and are labeled based on their influences, which are measured via the number of words in their Wiki pages.
- 3) *Aminer collaboration network* [35], [40]: This network represents the relationships of cooperation between academic authors. The original dataset contains 1 560 640 authors and 4 258 946 relationships, and we extract three connected subgraphs where the words “network,” “data,” and “system” are in their key terms (shortly denoted as Aminer-network, Aminer-data, and Aminer-system), respectively. Nodes represent authors and edges denote relationships, and the nodes are labeled by their H-indexes.

B. Baseline

We compare our model DRSR with eight start-of-the-art methods including classical and deep learning methods.

- 1) *Node2vec* [21]: This method designs two hyperparameters p and q to control the random walk for capturing both vertex proximity and structural similarity and leverages neural language model to learn node embedding.
- 2) *RoIX* [25]: This method is based on NMF on the feature matrix which is extracted via ReFeX [26].
- 3) *DRNE* [32]: This method leverages a layer normalized LSTM model to learn node embeddings recursively by assuming that the regular equivalent structure of a node has been encoded by the representations of its neighbors.
- 4) *GraphWave* [29]: This method treats the features of spectral graph wavelets as probability distributions, and generates embeddings via empirical characteristic functions.
- 5) *Struc2vec* [11]: It learns role-based embeddings by encoding structural identity of nodes via some sequences of walks on a reconstructed multilayer complete graph.
- 6) *Struc2gauss* [31]: This method applies RoleSim and optimizes a KL-energy function to learn Gaussian embeddings.
- 7) *GAS* [33]: It constructs the auto-encoder framework and leverages GCN on adjacency matrix to reconstruct some basic features of ReFeX.
- 8) *RESA* [35]: It leverages the variational auto-encoder on some structural features to reconstruct them and adds a regularizer of node degree to preserve local structure.

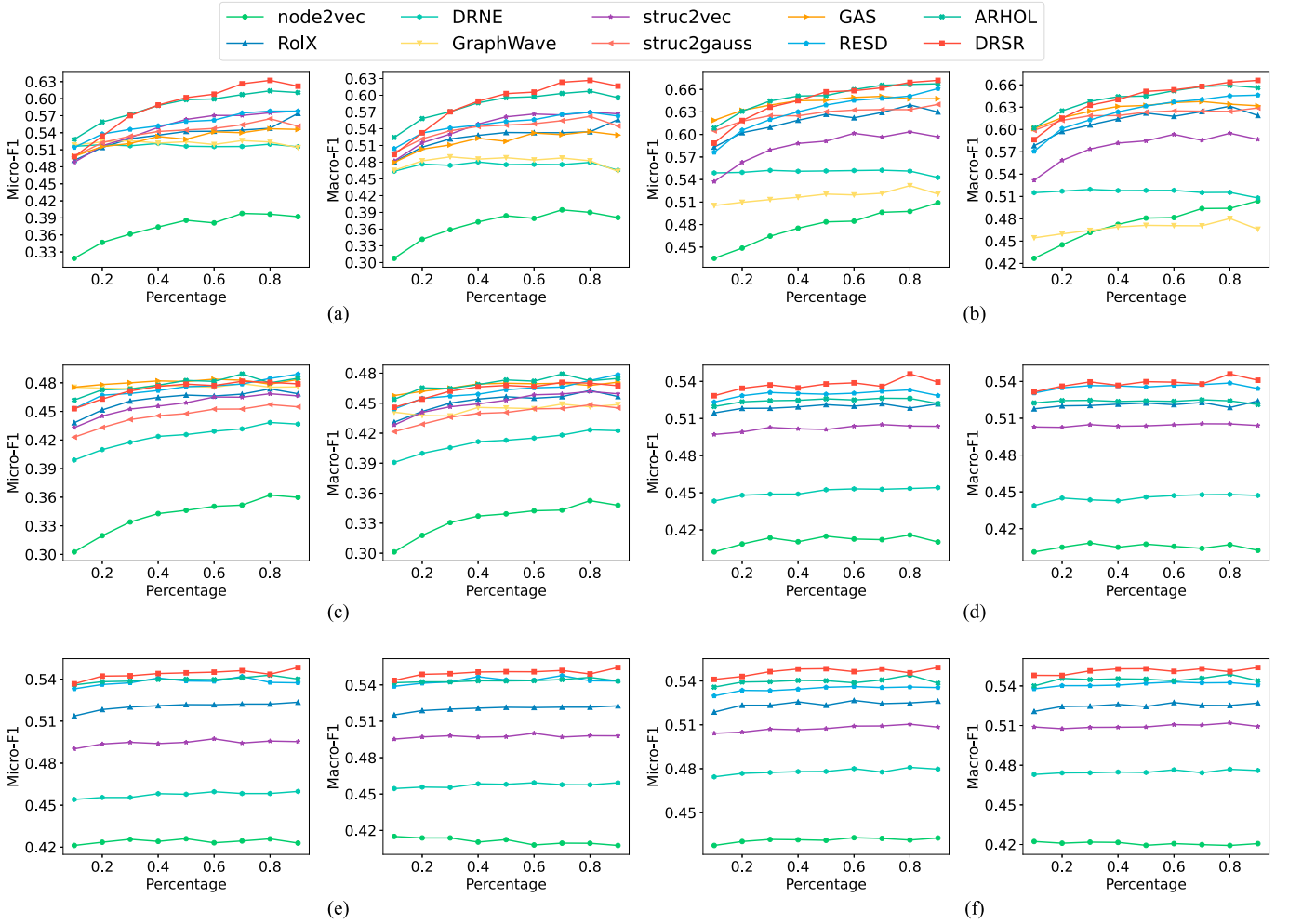


Fig. 3. Micro-F1 and macro-F1 scores on different real-world networks. The horizontal axis denotes the ratio of training samples. (a) Europe. (b) United States. (c) Actor. (d) Aminer-network. (e) Aminer-data. (f) Aminer-system.

- 9) *ARHOL* [13]: It integrates the local and higher-order features via an adversarial framework to generate structural node representations.

The dimension of GraphWave is set to 100, while the dimensions of other methods are set to 128. All the other parameters are tuned to their best performances.

C. Experiment Setting

For our model DRSR, we empirically use the ReLU as the activation function in both the encoder and decoder. We set the number of encoder layers to 2 and the decoder layer to 3, and the number of factors K is set to 8. Then for each factor k , the dimension of hidden layers is set to 32, and the dimension of output is set to 16, so the dimension of the node embeddings is 128. In the process of training DRSR, we set the max epochs to 500 with a learning rate 0.001. We use the Adam optimizer to update parameters.

All the experiments are conducted on a Linux server (Ubuntu 18.04.5 LTS) equipped with an Intel(R) Xeon(R) Silver 4210 CPU @ 2.20GHz, 256GB RAM and 2 NVIDIA 3090 GPUs. All models are implemented in PyTorch version 1.9.0, DGL version 0.7.1 with CUDA version 11.1, scikit-learn

version 0.19.0 and Python 3.6. The link to data and code is <https://anonymous.4open.science/r/DRSR-BAB3>.

D. Role Classification

In this section, we conduct the role classification experiment on several real-world networks, where nodes are labeled based on their true roles. The task of role-oriented network embedding is unsupervised, and we measure the performance of embedding methods via downstream tasks. As most of the GNN-based models are designed for supervised or semisupervised tasks, we don't compare with them in this section, but leverage some advanced GNNs to replace our encoder and analyze the influences in Section IV-G. In this experiment, we first generate node embeddings, then randomly choose different ratios of samples as the training set, while the remains are used for test. We train a linear logistic regression classifier and report the micro-F1 and macro-F1 scores as shown in Fig. 3. Without loss of generality, we repeat the process for 20 times and report the mean value. We have the following observations.

The network sizes in the six pictures are increasing, and for the small networks (Europe, United States, and Actor), we find that our model obtains significant improvement, especially in

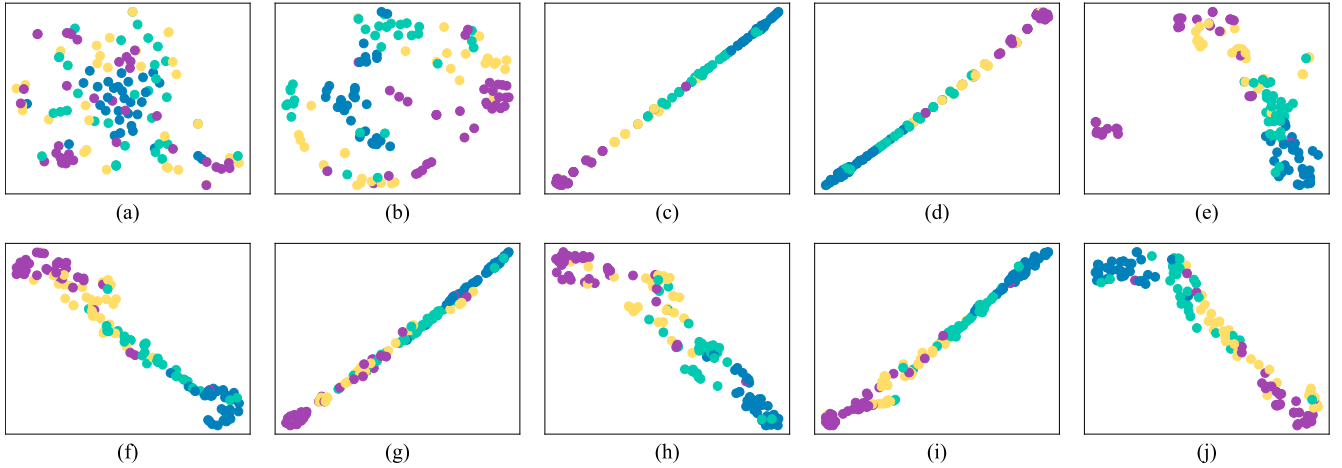


Fig. 4. Visualization on Brazil network. (a) Node2vec. (b) RolX. (c) DRNE. (d) GraphWave. (e) Struc2vec. (f) Struc2gauss. (g) GAS. (h) RESD. (i) ARHOL. (j) DRSR.

the Europe network. We also observe that the performance of node2vec is the poorest among all the baselines. The reason is that it still leverages the id-based random walk and cannot overcome its shortcomings, and the node representations generated by it fail to capture structural similarities. As for other baselines, RolX leverages the same features as us, while GAS only uses some basic features. However, the scores of RolX are much lower than ours, and GAS has the similar performance. This is because the NMF cannot fully use the network structure, and the GNN framework can make use of it. We also observe that GraphWave and GAS obtain similar scores as ours on Actor, but the macro-F1 score of our model DRSR is better than GAS.

As for the large networks (three Aminer collaboration networks), it is worth noting that GraphWave and GAS cannot obtain results because of out-of-memory error, while struc2gauss cannot generate node representations within 48 h. We observe that the classification scores become stable even if the ratio of training samples changes. The reason is that the network size is large enough, and a small ratio of samples can make the classifier converge. We find that DRSR always performs best. These results demonstrate the effectiveness of our model.

E. Visualization

We conduct the visualization experiment on the Brazil network to analyze whether these models can generate similar representations for nodes in the same roles. The dimensions of all the embeddings are as described in Section IV-B, and we use the T-distributed stochastic neighbor embedding (T-SNE) methods to reduce the dimensions to 2. As shown in Fig. 4, the first figure reports the network structure, and each point in the others denotes the node in the Brazil network, and the color represents its role. It is hoped that points of the same color should be clustered together, and those in different roles should be distant. We observe that points in node2vec are almost randomly distributed, indicating that it cannot capture role information. RolX and struc2gauss divide the purple nodes too far apart, while DRNE, GraphWave, and GAS may cause over-fitting to role equivalent because their embeddings are

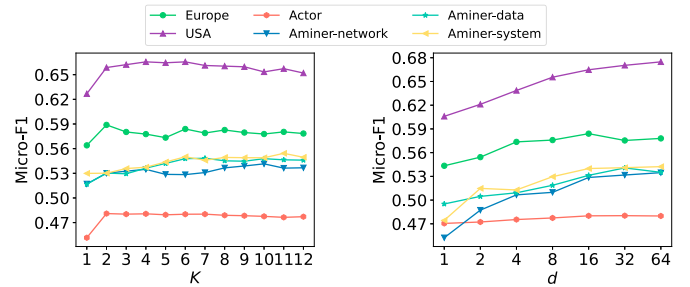


Fig. 5. Micro-F1 scores with respect to the number of latent factors K and dimension of representations d .

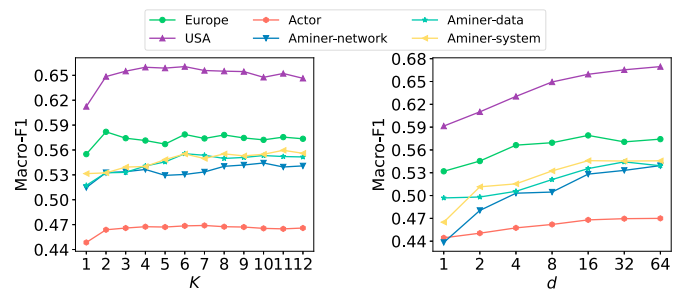


Fig. 6. Macro-F1 scores with respect to the number of latent factors K and dimension of representations d .

clustered into a line. Our model DRSR and struc2gauss have similar results, but points in DRSR are more clearly distinguished. The experiment indicates that our model can capture structural similarities to describe node roles.

F. Parameter Analysis

In this section, we analyze the influences of the number of latent factors K and dimension of representations d , and we report the micro-F1 and macro-F1 scores in Figs. 5 and 6. As shown in figures, we change the number of factors in the range [1, 12], and observe that the model performs best in Europe and United States at $K = 6$, while performs best in the three

TABLE III
MICRO-F1 SCORE OF DIFFERENT MODEL WITH 70% TRAINING SAMPLES

Model	Europe	United States	Actor	Aminer-Network	Aminer-Data	Aminer-System
GCN	0.5672	0.6304	0.4796	0.5322	0.5359	0.5425
GAT	0.5497	0.6311	0.4724	0.5205	0.5350	0.5313
DisenGCN	0.5506	0.6286	0.4710	0.4975	0.5025	0.5185
w/o Disen	0.5429	0.6302	0.4824	0.5267	0.5230	0.5333
w/o NC	0.5271	0.6134	0.4714	0.4983	0.4999	0.5219
DRSR	0.6262	0.6621	0.4816	0.5359	0.5461	0.5482

Note: The bold entries denote the best scores.

TABLE IV
MACRO-F1 SCORE OF DIFFERENT MODEL WITH 70% TRAINING SAMPLES

Model	Europe	United States	Actor	Aminer-Network	Aminer-Data	Aminer-System
GCN	0.5569	0.6246	0.4689	0.5373	0.5422	0.5469
GAT	0.5408	0.6252	0.4588	0.5182	0.5365	0.5342
DisenGCN	0.5389	0.6237	0.4543	0.4941	0.5023	0.5182
w/o Disen	0.5369	0.6165	0.4614	0.5219	0.5258	0.5318
w/o NC	0.5126	0.6026	0.4489	0.4962	0.4992	0.5212
DRSR	0.6234	0.6579	0.4708	0.5379	0.5519	0.5530

Note: The bold entries denote the best scores.

Aminer networks at $K = 10$. Both the micro-F1 and macro-F1 scores have the same trend. The reason is that they are the air-traffic and coauthor networks, respectively and have similar characters. As for the Actor network, we obtain the best results at $K = 2$. We also observe that the scores increase significantly from 1 to 2. The reason is the representations cannot obtain enough information of network structure. It also indicates that different networks possess different entangled factors, and our model can truly disentangle them to generate high-quality role-based node representations.

For the embedding dimension, we fix the number of factors and adjust the dimension for each k . As shown in the right part, we find that the performance improves with the dimension increasing and becomes stable at $d \geq 16$. Our model can generate node representation with proper dimensions and can be easily applied to downstream tasks.

G. Ablation Study

The main contribution of our model is the design of the disentangled GNN-based framework and the role-aware neighbor choosing mechanism. And to verify the correctness and effectiveness of our hypothesis, we replace our encoder with some popular GNN models (GCN [9], GAT [41], and DisenGCN [19]). As these GNN models are all designed for the semisupervised task, we leverage them as the encoders and use our decoder to reconstruct the same features. The number of factors in DisenGCN and our DRSR are set to be the same, the heads in GAT are also identical. Besides, we also remove some parts of our model and compare the performance. Specifically, the “w/o Disen” denotes our model without the disentangled encoder part, and the “w/o NC” denotes without neighbor choosing mechanism.

All the dimensions are set to 128, and we use 70% nodes to train the classifier with repeating 20 times. The results are reported in Tables III and IV. We mark the best scores in

bold fonts, and find that our model DRSR always performs best compared with these popular GNN encoders. Interestingly, DisenGCN has the ability to disentangle factors, but the scores are lower than classical GNNs, because it concentrates too much on proximity and only aggregates similar neighbors in each factor. And if without neighbor choosing (w/o NC), the performance declines a lot. The experiment demonstrates that our hypotheses are correct, and our model can disentangle the entangled patterns of neighbors to effectively capture roles.

H. Disentangling Synthetic Graphs

To further investigate the superiority of our disentangled framework, we construct a series of synthetic graphs with multiple entangled latent factor. We first generate a basic graph based on the United States dataset, where there are 1190 nodes and four classes. Then we randomly shuffle nodes and repeat this operation to generate K graphs. These network can be regarded as entangled by different latent factors (permutation action on nodes) with invariant network structure or attributes. Finally we generate the final synthetic graph by summing the adjacency matrices to make the K graphs entangle, so each node have K labels and there are 4000 classes for the final synthetic graph. Thus, we construct a task of multilabel classification to measure the effectiveness of disentanglement.

We report the macro-F1 scores in Table V. We choose the K from 3 to 8 to generate six synthetic graphs, and to keep the fairness, we tune the hyperparameter of our model k as usual. In other words, we make the hyperparameters unchanged for all these graphs. Note that we don’t compare with struc2gauss because it cannot obtain results within 48 h. We can observe that with the increase of K , this multilabel classification task becomes harder, as all methods exhibit a decreasing trend. Compared with other baselines, our model DRSR shows significant improvement. Although some methods such as RESD and ARHOL have may have similar scores in role classification

TABLE V
MACRO-F1 SCORES ON SYNTHETIC GRAPHS

Method	Number of Latent Factors					
	$k = 3$	$k = 4$	$k = 5$	$k = 6$	$k = 7$	$k = 8$
Node2vec	0.3763	0.3602	0.3472	0.3459	0.3517	0.3329
RoIX	0.3670	0.3319	0.3343	0.3246	0.3151	0.3191
DRNE	0.3665	0.3603	0.3433	0.3409	0.3425	0.3375
GraphWave	0.3645	0.3651	0.3447	0.3413	0.3402	0.3371
Struc2vec	0.3539	0.3332	0.3222	0.3218	0.3212	0.3134
GAS	0.3708	0.3578	0.3449	0.3454	0.3423	0.3358
RESN	0.3399	0.3255	0.3449	0.3164	0.3189	0.3119
ARHOL	0.3775	0.3607	0.3510	0.3164	0.3396	0.3298
DRSR	0.3967	0.3738	0.3605	0.3517	0.3444	0.3390

Note: The bold entries denote the best scores.

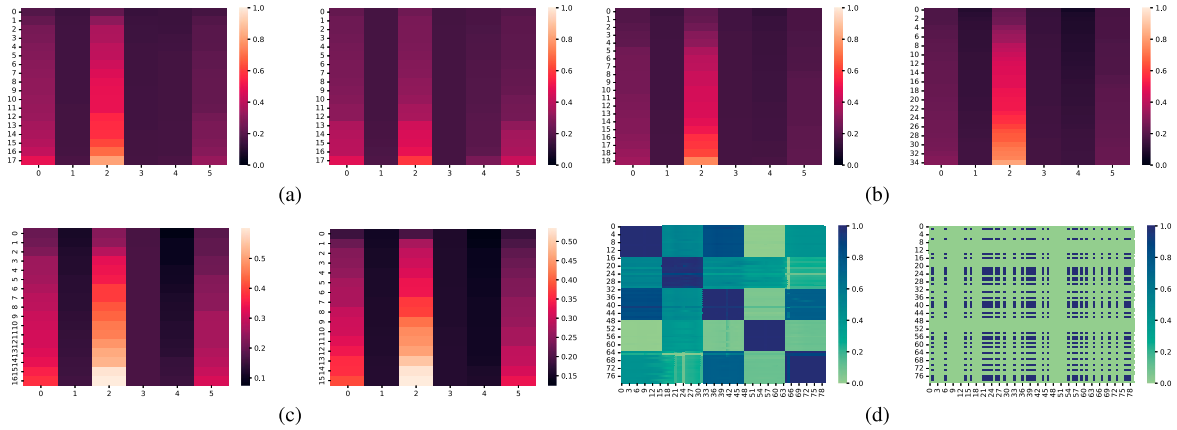


Fig. 7. Case study for disentangled role equivalence. (a) Nodes with same features but different roles. (b) Nodes with same roles but different features. (c) Nodes with same features and roles. (d) Correlation of representation dimensions on synthetic network.

experiment, they get lower scores in this task. The experiment demonstrates the superiority of our disentangled framework.

In addition, we conduct another experiment on synthetic networks to explore the disentanglement ability of our model. We leverage the SBM model to generate five random networks with 1000 nodes, where connection probability of nodes can be different within and between blocks, and we adjust the parameters p and q to make their average degrees similar. The final synthetic network is obtained by adding all the edges, while we extract structural features for each random graph and then combine them. We set $K = 5$ and report the correlation of representation dimensions on a synthetic network. As shown in Fig. 7(d), the first subfigure is obtained by DRSR while the other one is obtained via GCN. There are five clear blocks in our model, indicating that the disentangled representations have truly focused on different aspects with respect to different latent factors, but the GCN cannot handle it (other baselines have the same performance). It demonstrates that DRSR has the disentanglement ability.

I. Case Study

The main contribution of our model is that it extends the definition of role equivalence and can distinguish different roles

even if the local structures are the same. Thus, to exhibit the superiority of our model more clearly, we conduct the case study experiment, aiming to answer the following three questions. 1) Can DRSR distinguish nodes in different roles but have the same local structure? 2) Can DRSR find nodes in the same role but have different structures? 3) Can DRSR find nodes in the same role and the structures are also the same?

To answer the above questions, we choose some real cases in the Europe network to explore whether the role-aware neighbor choosing mechanism can improve the power of distinguishing roles. The roles of nodes are labeled by groundtruths, and for measuring local structures, we apply the ReFeX [26] methods, which extracts powerful local structural features for role discovery. Then to verify the correctness of our method, we leverage the interactive weights in the role-aware neighbor choosing mechanism to draw heatmaps.

1) Can DRSR Distinguish Nodes in Different Roles But Have the Same Local Structure?

The disentangled role equivalence is based on the hypothesis that $\mathbb{P}_k(u) = \mathbb{P}_k(v)$ for all $k = 1, 2, \dots, K$, meaning that nodes in the same role should interact with their neighbors in similar ways, so we draw a heatmap to compare whether the distributions of interactive weights of node u and v are similar. We choose two nodes with the same features but different roles, and

as shown in Fig. 7(a), the horizontal axis denotes the factors, and the vertical axis denotes the interactive weights, where lighter color denotes higher interactive weight. We can observe that the interactive weights of these two nodes are significantly different on channels 2 and 4, even if they have the same structural features. The result indicates that our model can disentangled complex relationships to discover roles.

2) Can DRSR Find Nodes in the Same Role But Have Different Structures?

For this question, we choose two nodes in the same role but have different structural features. For traditional role discovery methods, they will mistakenly assign them to different roles because of the different structural features, while our model can make great progress in this situation. As shown in Fig. 7(b), we observe that the distribution of interactive weights is similar (the vertical axis denotes the set of interactive weights) for all factors even if the structural features are different. It demonstrates the great power of our model DRSR.

3) Can DRSR Find Nodes in the Same Role and the Structures Are Also the Same?

Now that our model can disentangle the complex local structure and has far more power than other role discovery methods, it is still important to investigate the correctness in the traditional situation (same local structure and same role). Thus, we choose two nodes having the same structural features and same role and then compare their interactive weights. As shown in Fig. 7(c), we observe their interactive weights with neighbors are similar. All the results demonstrate the correctness of our hypothesizes and the superiority of our disentangled model.

V. CONCLUSION

In this article, we reinforce the definition of role equivalence. We assume that node roles are influenced by multiple latent factors with neighbors, and nodes in the same role should possess equivalent entangled patterns. To prove the correctness of these hypothesis, we propose a disentangled GNN model DRSR to learn role-based node representation. Specifically, we design a novel role-aware neighbor choosing mechanism which can adaptively assign different weights to neighbors in each latent factor to disentangle the interactive patterns related to roles of nodes. We conduct extensive experiments on real-world and synthetic networks, and compared with state-of-the-art methods including popular GNNs, the results demonstrate the effectiveness of our model.

REFERENCES

- [1] R. A. Rossi, D. Jin, S. Kim, N. K. Ahmed, D. Koutra, and J. B. Lee, "On proximity and structural role-based embeddings in networks: Misconceptions, techniques, and applications," *ACM Trans. Knowl. Discovery Data (TKDD)*, vol. 14, no. 5, pp. 1–37, 2020.
- [2] F. Lorrain and H. C. White, "Structural equivalence of individuals in social networks," *J. Math. Sociol.*, vol. 1, no. 1, pp. 49–80, 1971.
- [3] P. Jiao, X. Guo, T. Pan, W. Zhang, Y. Pei, and L. Pan, "A survey on role-oriented network embedding," *IEEE Trans. Big Data*, vol. 8, no. 4, pp. 933–952, Aug. 2022.
- [4] J. Qiu et al., "GCC: Graph contrastive coding for graph neural network pre-training," in *Proc. 26th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2020, pp. 1150–1160.
- [5] N. Ahmed et al., "Role-based graph embeddings," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 5, pp. 2401–2415, May 2022.
- [6] R. A. Rossi and N. K. Ahmed, "Role discovery in networks," *IEEE Trans. Knowl. Data Eng.*, vol. 27, no. 4, pp. 1112–1131, Apr. 2015.
- [7] J. Jin, M. Heimann, D. Jin, and D. Koutra, "Toward understanding and evaluating structural node embeddings," *ACM Trans. Knowl. Discovery Data (TKDD)*, vol. 16, no. 3, pp. 1–32, 2021.
- [8] B. Perozzi, R. Al-Rfou, and S. Skiena, "Deepwalk: Online learning of social representations," in *Proc. 20th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2014, pp. 701–710.
- [9] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–14.
- [10] Q. Tian et al., "Lower order information preserved network embedding based on non-negative matrix decomposition," *Inf. Sci.*, vol. 572, pp. 43–56, Sep. 2021.
- [11] L. F. Ribeiro, P. H. Saverese, and D. R. Figueiredo, "Struc2vec: Learning node representations from structural identity," in *Proc. 23rd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2017, pp. 385–394.
- [12] S. Salvador and P. Chan, "Toward accurate dynamic time warping in linear time and space," *Intell. Data Anal.*, vol. 11, no. 5, pp. 561–580, 2007.
- [13] W. Zhang et al., "Role-oriented network embedding based on adversarial learning between higher-order and local features," in *Proc. 30th ACM Int. Conf. Inf. Knowl. Manage.*, 2021, pp. 3632–3636.
- [14] P. Jiao, Q. Tian, W. Zhang, X. Guo, D. Jin, and H. Wu, "Role discovery-guided network embedding based on autoencoder and attention mechanism," *IEEE Trans. Cybern.*, vol. 53, no. 1, pp. 365–378, Jan. 2023.
- [15] H. Kim and A. Mnih, "Disentangling by factorising," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2018, pp. 2649–2658.
- [16] A. A. Alemi, I. Fischer, J. V. Dillon, and K. Murphy, "Deep variational information bottleneck," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–19.
- [17] F. Doshi-Velez and B. Kim, "Towards a rigorous science of interpretable machine learning," 2017, *arXiv:1702.08608*.
- [18] Y. Bengio, A. Courville, and P. Vincent, "Representation learning: A review and new perspectives," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 35, no. 8, pp. 1798–1828, 2013.
- [19] J. Ma, P. Cui, K. Kuang, X. Wang, and W. Zhu, "Disentangled graph convolutional networks," in *Proc. Int. Conf. Mach. Learn.*, PMLR, 2019, pp. 4212–4221.
- [20] J. Ma, C. Zhou, P. Cui, H. Yang, and W. Zhu, "Learning disentangled representations for recommendation," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 32, 2019, pp. 5711–5722.
- [21] A. Grover and J. Leskovec, "Node2vec: Scalable feature learning for networks," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2016, pp. 855–864.
- [22] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, 2013, pp. 3111–3119.
- [23] X. Ma, G. Qin, Z. Qiu, M. Zheng, and Z. Wang, "Riwalk: Fast structural node embedding via role identification," in *Proc. IEEE Int. Conf. Data Mining (ICDM)*, 2019, pp. 478–487.
- [24] D. Jin, M. Heimann, R. A. Rossi, and D. Koutra, "Node2bits: Compact time- and attribute-aware node representations for user stitching," in *Proc. Joint Eur. Conf. Mach. Learn. Knowl. Discovery Databases*, 2019, pp. 483–506.
- [25] K. Henderson et al., "RoIX: Structural role extraction & mining in large graphs," in *Proc. 18th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2012, pp. 1231–1239.
- [26] K. Henderson et al., "It's who you know: Graph mining using recursive structural features," in *Proc. 17th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2011, pp. 663–671.
- [27] S. Gilpin, T. Eliassi-Rad, and I. Davidson, "Guided learning for role discovery (GLRD) framework, algorithms, and applications," in *Proc. 19th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2013, pp. 113–121.
- [28] P. V. Gupte, B. Ravindran, and S. Parthasarathy, "Role discovery in graphs using global features: Algorithms, applications and a novel evaluation strategy," in *Proc. IEEE 33rd Int. Conf. Data Eng. (ICDE)*, 2017, pp. 771–782.
- [29] C. Donnat, M. Zitnik, D. Hallac, and J. Leskovec, "Learning structural node embeddings via diffusion wavelets," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2018, pp. 1320–1329.

- [30] G. Nikolentzos and M. Vazirgiannis, "Learning structural node representations using graph kernels," *IEEE Trans. Knowl. Data Eng.*, vol. 33, no. 5, pp. 2045–2056, May 2021.
- [31] Y. Pei, X. Du, J. Zhang, G. Fletcher, and M. Pechenizkiy, "Struc2gauss: Structural role preserving network embedding via Gaussian embedding," *Data Mining Knowl. Discovery*, vol. 34, pp. 1072–1103, May 2020.
- [32] K. Tu, P. Cui, X. Wang, P. S. Yu, and W. Zhu, "Deep recursive network embedding with regular equivalence," in *Proc. 24th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2018, pp. 2357–2366.
- [33] X. Guo, W. Zhang, W. Wang, Y. Yu, Y. Wang, and P. Jiao, "Role-oriented graph auto-encoder guided by structural information," in *Proc. Int. Conf. Database Syst. Adv. Appl.*, 2020, pp. 466–481.
- [34] H. Ke et al., "Deep mutual encode model for network embedding from structural identity," *IEEE Access*, vol. 7, pp. 177484–177496, 2019.
- [35] W. Zhang, X. Guo, W. Wang, Q. Tian, L. Pan, and P. Jiao, "Role-based network embedding via structural features reconstruction with degree-regularized constraint," *Knowl.-Based Syst.*, vol. 218, Apr. 2021, Art. no. 106872.
- [36] X. Guo, Q. Tian, W. Zhang, W. Wang, and P. Jiao, "Learning stochastic equivalence based on discrete Ricci curvature," in *Proc. 30th Int. Joint Conf. Artif. Intell. (IJCAI)*, 2021, pp. 1456–1462.
- [37] Y. Ollivier, "Ricci curvature of Markov chains on metric spaces," *J. Functional Anal.*, vol. 256, no. 3, pp. 810–864, 2009.
- [38] I. Higgins et al., "Beta-VAE: Learning basic visual concepts with a constrained variational framework," in *Proc. Int. Conf. Learn. Representations*, 2017, pp. 1–22.
- [39] J. Tang, J. Sun, C. Wang, and Z. Yang, "Social influence analysis in large-scale networks," in *Proc. 15th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2009, pp. 807–816.
- [40] J. Tang, J. Zhang, L. Yao, J. Li, L. Zhang, and Z. Su, "Arnetminer: Extraction and mining of academic social networks," in *Proc. 14th ACM SIGKDD Int. Conf. Knowl. Discovery Data Mining*, 2008, pp. 990–998.
- [41] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Lio, and Y. Bengio, "Graph attention networks," in *Proc. Int. Conf. Learn. Representations*, 2018, pp. 1–12.



Wang Zhang received the master's degree in computer science from Tianjin University, Tianjin, China, in 2022. He is currently working toward the Ph.D. degree in computer science with the School of Computer Science and Technology, Tianjin University.

His research interests include complex network analysis and embedding.



Lin Pan received the Ph.D. degree in computer science from Tianjin University, Tianjin, China, in 2016.

He is currently a Lecturer with the School of Marine Science and Technology, Tianjin University. His research interests include complex network analysis, social computing, and marine data mining.



Xuan Guo is currently working toward the Ph.D. degree in computer science with the College of Intelligence and Computing, Tianjin University, Tianjin, China.

His research interests include complex network analysis, role discovery, network representation learning, and percolation model.



Pengfei Jiao (Member, IEEE) received the Ph.D. degree in computer science from Tianjin University, Tianjin, China, in 2018.

From 2018 to 2021, he was a Lecturer with The Center for Bio-safety Research and Strategy of Tianjin University. He is currently a Professor with the School of Cyberspace, Hangzhou Dianzi University, Hangzhou, China. His research interests include complex network analysis and its applications.